

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра вычислительной техники (ВТ)

Отчёт

по дисциплине

«Разработка программно-аппаратного обеспечения информационных и автоматизированных систем»

Выполнил студент группы ИММО-01-25 Горбунов А.Р.

Принял старший преподаватель Дуксин Н.А.

Работа выполнена «15» декабря 2025 г.

«Зачтено» « » 2025 г.

Москва 2025

Введение

Целью данной лабораторной работы является практическое освоение методов проектирования цифровых вычислительных устройств на уровне регистровых передач, RTL, и изучение специфики аппаратной реализации алгоритмов. Ключевой задачей работы стала трансляция поведенческой модели учебного процессора, ранее разработанной на языке высокого уровня Python, в синтезируемое описание на языке Verilog HDL. Актуальность исследования обусловлена необходимостью понимания инженерами-разработчиками фундаментальных принципов работы аппаратного обеспечения, таких как синхронная логика, организация памяти и управление потоками данных, которые скрыты при программной эмуляции.

В ходе выполнения работы требовалось спроектировать модуль процессорного ядра, поддерживающий заданную систему команд и Гарвардскую архитектуру, а также создать верификационное окружение, testbench для подтверждения корректности работы устройства. Проверка осуществлялась на двух контрольных алгоритмах: вычислении суммы элементов вектора и свертке массивов, результаты которых сравнивались с эталонными значениями программного эмулятора.

Архитектура и принципы функционирования процессора

Проектируемая система базируется на Гарвардской архитектуре, что подразумевает физическое и логическое разделение памяти команд и памяти данных. Такое архитектурное решение позволяет оптимизировать вычислительный процесс за счет независимого доступа к инструкциям и операндам. В разработанной модели память данных (Data Memory) организована как массив 16-битных знаковых слов размером 128 ячеек. Этого объема достаточно для размещения входных векторов и результатов вычислений. Память инструкций (Program Memory) реализована в виде отдельных массивов для кода операции и операндов, что упрощает декодирование команд при загрузке из внешнего файла.

Особое внимание при проектировании уделялось арифметико-логическому устройству. Поскольку процессор должен поддерживать вычисления с отрицательными числами, что необходимо для задачи свертки, все регистры и массивы памяти были объявлены с использованием ключевого слова signed. Это обеспечивает корректную работу арифметических операций в дополнительном коде (two's complement), избавляя от необходимости ручной реализации знаковой логики. Регистровый файл включает 8 регистров общего назначения, используемых как для арифметических операций, так и для косвенной адресации памяти.

Система команд процессора включает инструкции пересылки данных (LOAD, STORE), арифметические операции (ADD, INC, DEC, MUL), операцию очистки (CLR) и управление потоком (JNZ). Формат команд является четырехадресным, где инструкция содержит код операции и три операнда, что позволяет гибко управлять источниками и приемниками данных.

Программная реализация RTL-модели

Разработка модуля процессора vm_cpu велась на языке Verilog. Логика работы устройства полностью синхронна и управляется глобальным тактовым сигналом clk. Внутри модуля реализован механизм загрузки микропрограммы: при инициализации симуляции происходит чтение файла mcode.txt, содержащего машинный код в десятичном формате. С помощью системных функций $fopen и $fscanf инструкции последовательно загружаются во внутренние массивы памяти программ. Это решение позволяет гибко менять исполняемый алгоритм без перекомпиляции Verilog-кода.

Основной цикл работы процессора описан в блоке always, чувствительном к переднему фронту тактового сигнала. На каждом такте происходит выборка инструкции по адресу счетчика команд (PC), ее декодирование и исполнение. Управляющая логика реализована с помощью конструкции case, которая мультиплексирует потоки данных в зависимости от

текущего кода операции. Например, при выполнении инструкции умножения MUL производится перемножение значений двух регистров-источников и запись результата в регистр назначения. Инструкции работы с памятью LOAD и STORE поддерживают сложную логику адресации, позволяя использовать как прямые константы, так и значения регистров в качестве адресов, что критично для итеративной обработки массивов.

Для верификации проекта был создан модуль test. В отличие от синтезируемого ядра, данный модуль выполняет роль испытательного стенда. В блоке initial testbench производит прямую инициализацию памяти данных (uut.mem), загружая в нее тестовые векторы для алгоритмов суммы и свертки. Это имитирует начальное состояние ОЗУ перед запуском программы. Также модуль test генерирует тактовый сигнал и сигнал сброса, а затем мониторит состояние внутренних регистров и ключевых ячеек памяти, автоматически останавливая симуляцию при появлении сигнала halted.

Верификация и анализ результатов моделирования

Верификация проводилась методом сквозного сравнения результатов выполнения программ на RTL-модели и на эталонном эмуляторе Python. Первым этапом тестировался алгоритм суммирования элементов вектора. В память данных был загружен массив чисел, и после выполнения программы результат ожидался в ячейке памяти с адресом 39 (рис. 1).

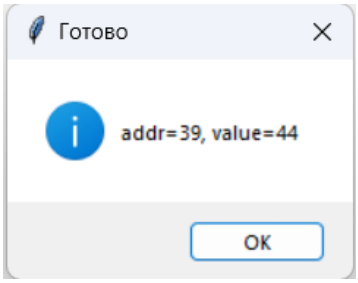


Рисунок 1 – Результат работы алгоритма суммы векторов на программной модели, реализованной на Python

При запуске RTL-симуляции в среде Vivado программа корректно отработала заданное количество циклов. Анализ временных диаграмм показал, что регистры-счетчики корректно декрементировались, а аккумулятор накапливал сумму. По завершении работы программы в ячейке памяти MEM39 было зафиксировано значение 44, что полностью совпадает с эталоном (рис. 2).

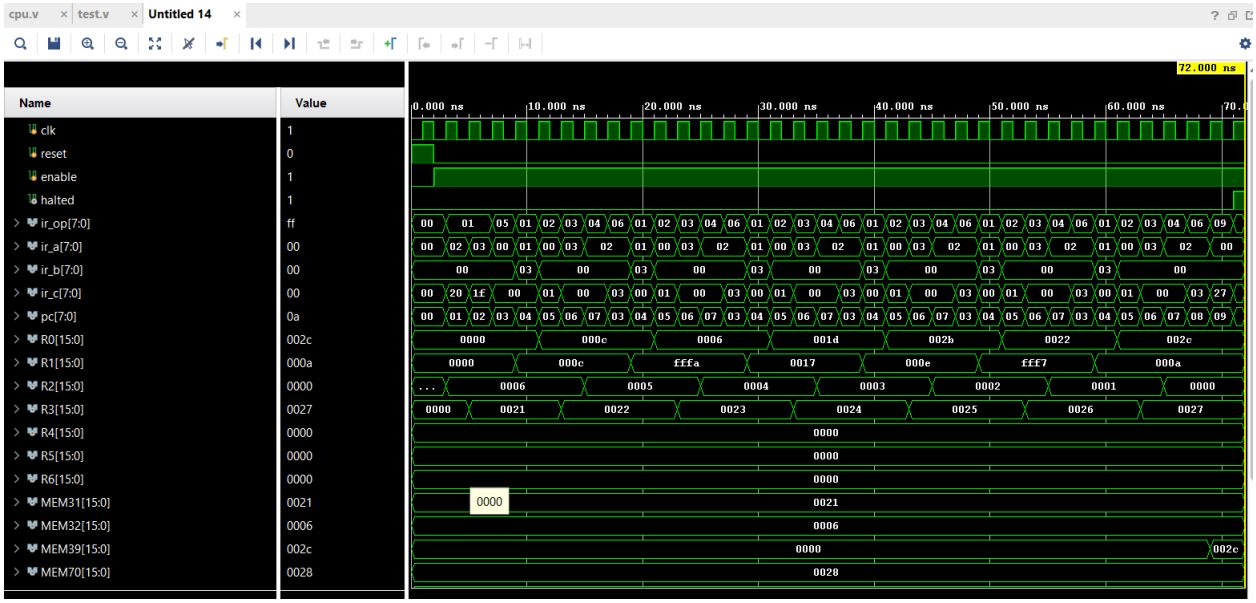


Рисунок 2 – Временная диаграмма RTL-симуляции процессора при выполнении задачи суммирования

Вторым этапом проверялся алгоритм свертки массивов, представляющий собой более высокую вычислительную нагрузку. Ожидаемый результат свертки, полученный на эмуляторе, составил -75, и должен был располагаться в ячейке памяти 90 (рис. 3).

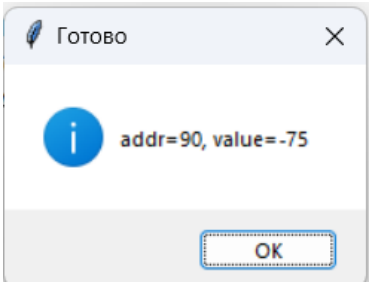


Рисунок 3 – Результат работы алгоритма свертки массивов на программной модели, реализованной на Python

В ходе симуляции Verilog-модели в ячейку MEM90 было записано значение, которое на осциллограммах отображалось как шестнадцатеричное

число. Учитывая знаковую природу арифметики, анализ битового представления подтвердил, что записанное значение соответствует десятичному числу -75. Это доказывает корректность реализации инструкции умножения MUL и правильную обработку переполнений и знаковых битов в тракте данных процессора (рис. 4-5).

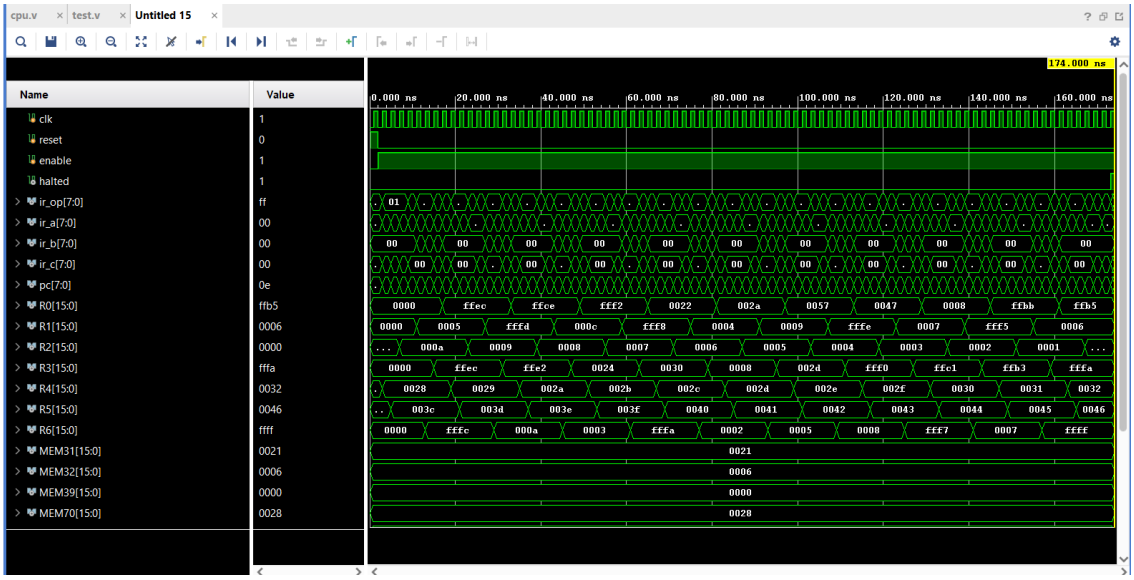


Рисунок 4 – Детализованная осциллограмма работы процессора в задаче свертки

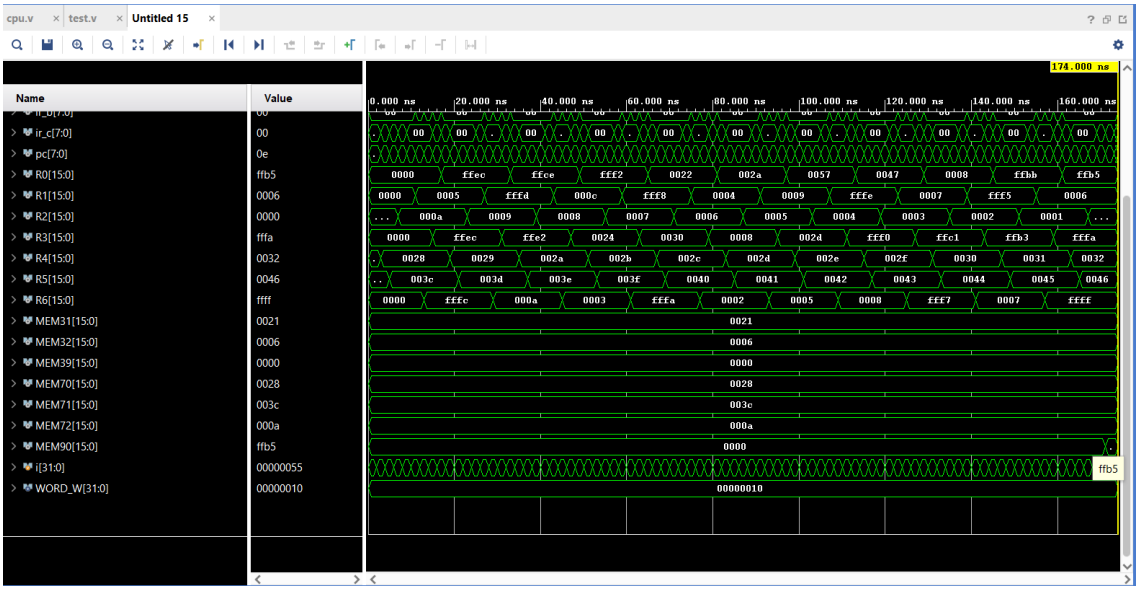


Рисунок 5 – Продолжение осциллограммы работы процессора в задаче свертки

Заключение

В результате выполнения лабораторной работы была успешно решена задача перевода программной модели процессора в аппаратное описание на языке Verilog. Созданный модуль vm_cpu демонстрирует корректную работу всех инструкций, заложенных в систему команд, включая операции управления циклом и знаковую арифметику.

Сравнительный анализ результатов моделирования подтвердил полную идентичность поведения RTL-описания и эталонного эмулятора на Python. Реализованный подход с раздельной загрузкой кода программы из файла и инициализацией данных через testbench доказал свою эффективность для отладки и верификации цифровых устройств. Полученное описание может быть использовано для дальнейшего синтеза на ПЛИС.

Приложение

Листинг 1. Модель процессора (vm_cpu)

```
`timescale 1ns/1ps

module vm_cpu #(
parameter integer MEM_SIZE  = 128,
parameter integer REG_N     = 8,
parameter integer WORD_W    = 16,
parameter integer PROG_SIZE = 256,
parameter [1023:0] PROG_FILE = "mcode.txt"
)(
input  wire clk,
input  wire reset,
input  wire enable,
output reg  halted,
output reg  [7:0] ir_op,
output reg  signed [7:0] ir_a,
output reg  signed [7:0] ir_b,
output reg  signed [7:0] ir_c,
output reg  [$clog2(PROG_SIZE)-1:0] pc
);

reg signed [WORD_W-1:0] mem [0:MEM_SIZE-1];
reg signed [WORD_W-1:0] r   [0:REG_N-1];

reg signed [7:0] prog_op [0:PROG_SIZE-1];
reg signed [7:0] prog_a  [0:PROG_SIZE-1];
reg signed [7:0] prog_b  [0:PROG_SIZE-1];
reg signed [7:0] prog_c  [0:PROG_SIZE-1];
integer prog_len;

integer fd, n;
integer t_op, t_a, t_b, t_c;
integer i;

initial begin
     for (i = 0; i < MEM_SIZE;  i = i + 1) mem[i] = 0;
     for (i = 0; i < REG_N;     i = i + 1) r[i]   = 0;
     for (i = 0; i < PROG_SIZE; i = i + 1) begin
     prog_op[i] = 0; prog_a[i] = 0; prog_b[i] = 0; prog_c[i] = 0;
     end

     ir_op = 0; ir_a = 0; ir_b = 0; ir_c = 0;
     pc = 0;
     halted = 0;
     prog_len = 0;

     fd = $fopen(PROG_FILE, "r");
     if (fd != 0) begin
     while (!$feof(fd) && prog_len < PROG_SIZE) begin
            n = $fscanf(fd, "%d %d %d %d", t_op, t_a, t_b, t_c);
            if (n == 4) begin
                    prog_op[prog_len] = t_op[7:0];
                    prog_a [prog_len] = t_a [7:0];
                    prog_b [prog_len] = t_b [7:0];
                    prog_c [prog_len] = t_c [7:0];
                    prog_len = prog_len + 1;
            end else begin
                    n = $fscanf(fd, "%*[^\n]\n");
```

```
            end
        end
        $fclose(fd);
        end
    end

    integer addr;
    integer a_i, b_i, c_i;

    function automatic signed [WORD_W-1:0] sx8(input signed [7:0] x);
        sx8 = {{(WORD_W-8){x[7]}}, x[7:0]};
    endfunction

    always @(posedge clk) begin
        if (reset) begin
        for (i = 0; i < REG_N; i = i + 1) r[i] <= 0;
        pc     <= 0;
        ir_op  <= 0;
        ir_a   <= 0;
        ir_b   <= 0;
        ir_c   <= 0;
        halted <= 0;
        end else if (enable && !halted) begin
        if (pc >= prog_len) begin
               halted <= 1'b1;
        end else begin
               ir_op <= prog_op[pc];
               ir_a  <= prog_a [pc];
               ir_b  <= prog_b [pc];
               ir_c  <= prog_c [pc];

               a_i = prog_a[pc];
               b_i = prog_b[pc];
               c_i = prog_c[pc];

               pc <= pc + 1'b1;

               case (prog_op[pc])
                       8'd0: begin
                       end

                       8'd1: begin
                       if (b_i == -1) begin
                            r[a_i] <= sx8(c_i[7:0]);
                       end else if (b_i == 0) begin
                            addr = c_i;
                            r[a_i] <= mem[addr];
                       end else begin
                            addr = r[b_i];
                            r[a_i] <= mem[addr];
                       end
                       end

                       8'd2: begin
                       // ADD
                       r[a_i] <= r[b_i] + r[c_i];
                       end

                       8'd3: begin
                       // INC
                       r[a_i] <= r[a_i] + 1;
                       end
```

```
                            8'd4: begin
                            // DEC
                            r[a_i] <= r[a_i] - 1;
                            end

                            8'd5: begin
                            // CLR
                            r[a_i] <= 0;
                            end
        8'd6: begin
                        // JNZ Ra, c
                        if (r[a_i] != 0) begin
                            pc <= c_i[$clog2(PROG_SIZE)-1:0];
                        end
                    end

                    8'd7: begin
                        // MUL
                        r[a_i] <= r[b_i] * r[c_i];
                    end

                    8'd9: begin
                        // STORE
                        if (b_i == 0) begin
                            addr = c_i;
                        end else begin
                            addr = r[b_i];
                        end
                        mem[addr] <= r[a_i];
                    end

                    8'd255: begin
                        // HALT
                        halted <= 1'b1;
                    end

                    default: begin
                        halted <= 1'b1;
                    end
                endcase
            end
        end
    end

    endmodule
```

Листинг 2. Модуль тестирования (test)

```
    `timescale 1ns/1ps

    module test;

        reg clk = 0;
        reg reset = 1;
        reg enable = 0;

        wire halted;
        wire [7:0] ir_op;
        wire signed [7:0] ir_a, ir_b, ir_c;
        wire [7:0] pc;

        vm_cpu #(
```

```
        .PROG_FILE("mcode.txt")
    ) uut (
        .clk(clk),
        .reset(reset),
        .enable(enable),
        .halted(halted),
        .ir_op(ir_op),
        .ir_a(ir_a),
        .ir_b(ir_b),
        .ir_c(ir_c),
        .pc(pc)
    );

    always #1 clk = ~clk;

    localparam integer WORD_W = 16;

    wire signed [WORD_W-1:0] R0 = uut.r[0];
    wire signed [WORD_W-1:0] R1 = uut.r[1];
    wire signed [WORD_W-1:0] R2 = uut.r[2];
    wire signed [WORD_W-1:0] R3 = uut.r[3];
    wire signed [WORD_W-1:0] R4 = uut.r[4];
    wire signed [WORD_W-1:0] R5 = uut.r[5];
    wire signed [WORD_W-1:0] R6 = uut.r[6];

    wire signed [WORD_W-1:0] MEM31 = uut.mem[31];
    wire signed [WORD_W-1:0] MEM32 = uut.mem[32];
    wire signed [WORD_W-1:0] MEM39 = uut.mem[39];

    wire signed [WORD_W-1:0] MEM70 = uut.mem[70];
    wire signed [WORD_W-1:0] MEM71 = uut.mem[71];
    wire signed [WORD_W-1:0] MEM72 = uut.mem[72];
    wire signed [WORD_W-1:0] MEM90 = uut.mem[90];

    integer i;

    initial begin
        for (i = 0; i < 128; i = i + 1)
            uut.mem[i] = 0;

        uut.mem[31] = 33;
        uut.mem[32] = 6;
        uut.mem[33] = 12;
        uut.mem[34] = -6;
        uut.mem[35] = 23;
        uut.mem[36] = 14;
        uut.mem[37] = -9;
        uut.mem[38] = 10;
        uut.mem[39] = 0;

        uut.mem[70] = 40;
        uut.mem[71] = 60;
        uut.mem[72] = 10;
        uut.mem[90] = 0;

        uut.mem[40] = 5;   uut.mem[41] = -3;  uut.mem[42] = 12;  uut.mem[43]
= -8;  uut.mem[44] = 4;
        uut.mem[45] = 9;   uut.mem[46] = -2;  uut.mem[47] = 7;   uut.mem[48]
= -11; uut.mem[49] = 6;

        uut.mem[60] = -4;  uut.mem[61] = 10;  uut.mem[62] = 3;   uut.mem[63]
= -6;  uut.mem[64] = 2;
```

```
        uut.mem[65] = 5;   uut.mem[66] = 8;   uut.mem[67] = -9;  uut.mem[68]
= 7;   uut.mem[69] = -1;

        #2 reset = 0;
        enable = 1;

        for (i = 0; i < 400; i = i + 1) begin
            #2;
            if (halted) begin
                $display("DONE: pc=%0d   MEM39=%0d   MEM90=%0d", pc, MEM39,
MEM90);
                $stop;
            end
        end

        $display("TIMEOUT:  pc=%0d  halted=%0d   MEM39=%0d  MEM90=%0d",  pc,
halted, MEM39, MEM90);
        $stop;
    end

    always @(posedge clk) begin
        if (!reset && enable && !halted) begin
            $display("t=%0t pc=%0d op=%0d a=%0d b=%0d c=%0d R0=%0d R1=%0d
R2=%0d R3=%0d R4=%0d R5=%0d R6=%0d MEM39=%0d MEM90=%0d",
                     $time, pc, ir_op, ir_a, ir_b, ir_c,
                     R0, R1, R2, R3, R4, R5, R6,
                     MEM39, MEM90);
        end
    end

endmodule
```